

SGP4 Software Tutorial

by **Dr. T.S. Kelso**
2023 Nov 25
Updated 2026 Jul 26

Background

We've had GP (General Perturbations) orbital data on CelesTrak for over 40 years now [see my [keynote address to SMOPS-2023](#) if you're curious to see how that came to pass]. Initially, it was only available for a small handful of satellites and had to be typed in by hand, but today we get data for tens of thousands of payloads and associated debris (e.g., dead satellites, rocket bodies, and bits from breakups and collisions), all available via the Internet. To use that data and get the best results, however, users must be sure to use a valid implementation of [SGP4](#).

SGP4 is a semi-analytic orbit propagator, which is a blend of a purely closed-form analytical representation of an orbital force model (a set of all the forces acting on a satellite), that typically has low fidelity (accuracy), when compared to a full-fidelity numerical integrator, that requires significantly more computation. Back in the 1970s, when SGP4 was being developed, it was important to balance the accuracy of the propagator with the time required to produce those results—especially given the limitations of computational resources at the time.

SGP4 does this by using doubly-averaged Brouwer mean elements in the GP (General Perturbations) element sets to provide the zeroth-order terms in a series expansion. Those terms are found by fitting observations with SGP4 (using a differential corrector process) in a way to minimize the overall residuals (differences between the observations and calculated observations). That means same version of SGP4 **must** be used with the GP data to produce the best results. After all, you wouldn't use the coefficients from a 3rd-order polynomial fit in a 2nd-order polynomial and expect to get good results, right?

That's probably more detail than you wanted to know, but if you accept that we need to use a valid version of SGP4 (the same one used to generate the GP data), you've got the main point!

The nice part of having a semi-analytic propagator is that we have a nearly closed-form way to calculate the satellite position and velocity as a function of time. And that means it is quite computationally efficient.

Verification & Validation

We've used the term "valid" implementation (or version) of SGP4 several times now, but just what does that mean? In testing, "validation" typically means that the model results match reality to the level of accuracy required for a particular application. What we are really talking about here is actually considered model "verification"—showing that the model results match the expected results. So, we are actually saying you need to ensure you have a "verified" implementation of SGP4, but that would probably cause more confusion.

So, how do we verify our model results? Well, since the US Space Force (USSF) maintains the operational version of SGP4 used to generate the GP data produced by 18 SDS, we need to be able to match their model outputs.

Fortunately, USSF has released an [official version](#) [requires being logged in on an account on [space-track.org](#)] that can be used to verify any other implementation of SGP4. While it is a black box—no source code is provided—it can generate position and velocity data that can be compared to output from our implementation.

To perform our verification, I implemented the C# version of SGP4Lib released with the paper ["Revisiting Spacetrack Report #3"](#) and the USSF version (SGP4Prop) in Microsoft Visual Studio and calculated position and velocity to 7 decimal places (0.1 mm and 0.1 mm/s) for the entire GP catalog (23,781 objects with GP data from 2022 Jul 20) at one-minute timesteps for ±1,440 minutes (±1 day). When comparing the results, I found the average maximum error for all objects to be less than 0.1 mm for position and 0.1 mm/s for velocity (roundoff error).

That's great, but that's a Windows implementation and we are looking for a cross-platform implementation in JavaScript that can be used by any CelesTrak user to verify another implementation (e.g., Python). So, I decided to start with [satellite-js](#), which is based on our code from "Revisiting Spacetrack Report #3." I used it to produce the same output and compared that to our results from SGP4Lib and found that the results only matched to the meter level or 3 decimal points for all objects.

The first thing I looked for was a common configuration error seen in many SGP4 implementations. While there are options in our SGP4Lib code to use the WGS72 or WGS84 variables, it is important to ensure that the WGS72 variables are used, since that is what is still used operationally by USSF today. While it seems logical that WGS84 would be used over 40 years after its release, one should not simply assume it is. In fact, continuing to use WGS72 makes some sense because there would be no way to know which GP data was produced using WGS72 and which used WGS84. Fixing that misconfiguration resulted in matches for all but 377 cases.

```
/* WRONG VALUES (WGS84)
var mu = 398600.5; // in km3 / s2
var earthRadius = 6378.137; // in km
var j2 = 0.00108262998905;
var j3 = -0.00000253215306;
var j4 = -0.00000161098761;
*/
// CORRECT VALUES (WGS72)
var mu = 398600.8; // in km3 / s2
var earthRadius = 6378.135; // in km
var j2 = 0.001082616;
var j3 = -0.00000253881;
var j4 = -0.00000165597;
```

The remaining cases were all for objects that had highly eccentric semi-synchronous orbits. There is only one section of the SGP4Lib code specifically dedicated to these types of orbits—labeled "near - half-day resonance terms" with 7 lines of code—so it was easy to compare the satellite-js code to our SGP4Lib code and find that there was a missing set of parentheses in satellite-js. Once that oversight was corrected, the results from satlite-js matched those from SGP4Lib.

```
// ----- near - half-day resonance terms -----
xomi = argpo + argpdot * atime;
x2omi = xomi + xomi;
x2li = xli + xli;
xndt = d2201 * Math.sin(x2omi + xli - g22) + d2211 * Math.sin(xli - g22) +
d3210 * Math.sin(xomi + xli - g32) + d3222 * Math.sin(-xomi + xli - g32) +
d4410 * Math.sin(x2omi + x2li - g44) + d4422 * Math.sin(x2li - g44) +
d5220 * Math.sin(xomi + xli - g52) + d5232 * Math.sin(-xomi + xli - g52) +
d5421 * Math.sin(xomi + x2li - g54) + d5433 * Math.sin(-xomi + x2li - g54);
xldot = xni + xfact;
xnddt = d2201 * Math.cos(x2omi + xli - g22) + d2211 * Math.cos(xli - g22) +
d3210 * Math.cos(xomi + xli - g32) + d3222 * Math.cos(-xomi + xli - g32) +
d5220 * Math.cos(xomi + xli - g52) + d5232 * Math.cos(-xomi + xli - g52) +
2.0 * (d4410 * Math.cos(x2omi + x2li - g44) + d4422 * Math.cos(x2li - g44) +
d5421 * Math.cos(xomi + x2li - g54) + d5433 * Math.cos(-xomi x2li - g54)); //
xnddt *= xldot;
```

Implementation

Now that we have verified [our implementation of satellite-js](#) produces the same output as the official USSF version of SGP4, how do we use it? Let's look at a couple of examples to see how easy that is to do.

First, we're going to take [our implementation of satellite-js](#), that knows how to ingest TLE-formatted data, and extend it to be able to ingest both CSV- and JSON-formatted data, as well. This is actually quite easy and only requires adding routines for each.

Whenever we use SGP4, the first thing we want to do is convert our GP data to the internal satrec format used by our code. For TLE-formatted data, that is done by the function [twoline2satrec\(\)](#). The purpose of twoline2satrec() is to (1) parse the text data and convert it to satrec properties and (2) to initialize the satrec record.

Parsing the data occurs between setting the values of satrec.satnum and satrec.no—in just 12 lines of code. Parsing the TLE format is quite complex, due to its nonstandard mathematical notation, but this code deconstructs Line 1 and Line 2, as needed. These 12 lines are all we need to change to create functions to ingest CSV- or JSON-formatted data.

So, [csv2satrec\(\)](#) starts simply by splitting the input line at the commas to create a string array (field). Then it is a simple matter of assigning each element of the array to the appropriate satrec property. If the property expects a float, we use parseFloat() to convert it.

```
function csv2satrec(csv) {
  var field = csv.split(',');
  var opsmode = 'a';
  var xpdotp = 1440.0 / (2.0 * pi); // 229.1831180523293;
  var satrec = {};
  satrec.error = 0;
  satrec.satnum = field[11]; // 11 = NORAD_CAT_ID
  satrec.epoch = new Date(field[2] + "Z"); // Date object, 2 = EPOCH, UTC
  var year = satrec.epoch.getUTCFullYear();
  satrec.epochyr = year % 100;
  var dayZeroUTC = new Date(Date.UTC(year, 0, 0));
  satrec.epochdays = (satrec.epoch - dayZeroUTC) / 86400000.0; // days
  satrec.ndot = parseFloat(field[15]); // 15 = MEAN_MOTION_DOT
  satrec.nddot = parseFloat(field[16]); // 16 = MEAN_MOTION_DDOT
  satrec.bstar = parseFloat(field[14]); // 14 = BSTAR
  satrec.inclo = parseFloat(field[5]); // 5 = INCLINATION
  satrec.nodeo = parseFloat(field[6]); // 6 = RA_OF_ASC_NODE
  satrec.ecco = parseFloat(field[4]); // 4 = ECCENTRICITY
  satrec.argpo = parseFloat(field[7]); // 7 = ARG_OF_PERICENTER
  satrec.mo = parseFloat(field[8]); // 8 = MEAN_ANOMALY
  satrec.no = parseFloat(field[3]); // 3 = MEAN_MOTION
```

There are a couple of other things to note here. We can actually directly ingest the ISO 8601-1 date/time format used in the CSV-formatted (and JSON-formatted) data using standard JavaScript. We will see how this simplifies many time calculations later. And because we use a full 4-digit year now, we don't need the Y2K workaround for 1957-2056.

```
satrec.epoch = new Date(field[2]); // Date object, 2 = EPOCH
```

But there is a hidden issue here when using JavaScript, because it only handles time to the millisecond level. That is overkill for most uses of SGP4, except that a roundoff error of 0.5 ms for objects in LEO traveling at 7.5 km/sec shows up as a difference of as much as 3.75 m.

So, we modified the csv2satrec() function to correct this by manually extracting the microseconds (if present) and using that to correct the epoch:

```
function csv2satrec(csv) {
  var field = csv.split(',');
  var opsmode = 'a';
  var xpdotp = 1440.0 / (2.0 * pi); // 229.1831180523293;
  var satrec = {};
  satrec.error = 0;
  satrec.satnum = field[11]; // 11 = NORAD_CAT_ID
  // Modifications
  var epoch = field[2]; // 2 = EPOCH, UTC
  console.log(epoch);
  satrec.epoch = new Date(epoch + "Z"); // Date object, UTC, ms precision
  satrec.microSeconds = epoch.includes(".") ? parseInt(epoch.split(".")[1].replace(
  console.log(satrec.microSeconds);
  var year = satrec.epoch.getUTCFullYear();
  satrec.epochyr = year % 100;
  var dayZeroUTC = new Date(Date.UTC(year, 0, 0));
  satrec.epochdays = ((satrec.epoch - dayZeroUTC) + satrec.microSeconds/1000.0) /
  console.log(satrec.epochdays);
  // End modifications
  satrec.ndot = parseFloat(field[15]); // 15 = MEAN_MOTION_DOT
  satrec.nddot = parseFloat(field[16]); // 16 = MEAN_MOTION_DDOT
  satrec.bstar = parseFloat(field[14]); // 14 = BSTAR
  satrec.inclo = parseFloat(field[5]); // 5 = INCLINATION
  satrec.nodeo = parseFloat(field[6]); // 6 = RA_OF_ASC_NODE
  satrec.ecco = parseFloat(field[4]); // 4 = ECCENTRICITY
  satrec.argpo = parseFloat(field[7]); // 7 = ARG_OF_PERICENTER
  satrec.mo = parseFloat(field[8]); // 8 = MEAN_ANOMALY
  satrec.no = parseFloat(field[3]); // 3 = MEAN_MOTION
```

That produces the same value of satrec.epochdays as calculated in twoline2satrec() (and satrec.jdsatepoch further down), but still does not produce the same verification output—likely because some other calculation is being truncated at the millisecond level. We are still working on this issue.

And [json2satrec\(\)](#) is even simpler in many ways. Instead of passing individual lines of data, we will take the entire JSON data element—in JSON or JSON-PRETTY format—and convert the entire thing to a JSON object using JavaScript's JSON.parse() command.

So, if

```
http_response = getData(query);
var json_data = http_response;
var json_obj = JSON.parse(json_data);
// Initialize JSON data
satrec = satellite.json2satrec(json_obj[0]);
```

The relevant changes for json2satrec() are:

```
satrec.satnum = jsonObj.NORAD_CAT_ID;
satrec.epoch = new Date(jsonObj.EPOCH + "Z"); // UTC
var year = satrec.epoch.getUTCFullYear();
satrec.epochyr = year % 100;
var dayZeroUTC = new Date(Date.UTC(year, 0, 0));
satrec.epochdays = (satrec.epoch - dayZeroUTC) / 86400000.0; // days
satrec.ndot = jsonObj.MEAN_MOTION_DOT;
satrec.nddot = jsonObj.MEAN_MOTION_DDOT;
satrec.bstar = jsonObj.BSTAR;
satrec.inclo = jsonObj.INCLINATION;
satrec.nodeo = jsonObj.RA_OF_ASC_NODE;
satrec.ecco = jsonObj.ECCENTRICITY;
satrec.argpo = jsonObj.ARG_OF_PERICENTER;
satrec.mo = jsonObj.MEAN_ANOMALY;
satrec.no = jsonObj.MEAN_MOTION;
```

Note that there is no need to use parseFloat() as we did in csv2satrec(), if the JSON data is properly formatted—including the JSON-formatted data provided by Space Track is not, since it puts all values—unfortunately those for numbers—in quotes. That means they will be interpreted as strings by JSON and have to be converted using parseFloat(). CelesTrak does not treat numbers (including the NORAD_CAT_ID or catalog number) as strings, as seen in [this example](#) for the ISS:

```
[{
  "OBJECT_NAME": "ISS (ZARYA)",
  "OBJECT_ID": "1998-067A",
  "EPOCH": "2026-06-19T12:16:41.638656",
  "MEAN_MOTION": 15.49315858,
  "ECCENTRICITY": 0.00045965,
  "INCLINATION": 51.6332,
  "RA_OF_ASC_NODE": 288.5889,
  "ARG_OF_PERICENTER": 285.0015,
  "MEAN_ANOMALY": 155.0751,
  "EPHEMERIS_TYPE": 0,
  "CLASSIFICATION_TYPE": "U",
  "NORAD_CAT_ID": 25544,
  "ELEMENT_SET_NO": 999,
  "REV_AT_EPOCH": 57211,
  "BSTAR": 0.0001560528,
  "MEAN_MOTION_DOT": 8.252e-5,
  "MEAN_MOTION_DDOT": 0
}]
```

Here is [the same data](#) from Space Track:

```
[
  {
    "CCSDS_OMM_VERSION": "3.0",
    "COMMENT": "GENERATED VIA SPACE-TRACK.ORG API",
    "CREATION_DATE": "2026-06-19T17:59:21",
    "ORIGINATOR": "18 SPCS",
    "OBJECT_NAME": "ISS (ZARYA)",
    "OBJECT_ID": "1998-067A",
    "CENTER_NAME": "EARTH",
    "REF_FRAME": "TEME",
    "TIME_SYSTEM": "UTC",
    "MEAN_ELEMENT_THEORY": "SGP4",
    "EPOCH": "2026-06-19T12:16:41.638656",
    "MEAN_MOTION": "15.49315858",
    "ECCENTRICITY": "0.00045965",
    "INCLINATION": "51.6332",
    "RA_OF_ASC_NODE": "288.5889",
    "ARG_OF_PERICENTER": "285.0015",
    "MEAN_ANOMALY": "155.0751",
    "EPHEMERIS_TYPE": "0",
    "CLASSIFICATION_TYPE": "U",
    "NORAD_CAT_ID": "25544",
    "ELEMENT_SET_NO": "999",
    "REV_AT_EPOCH": "57211",
    "BSTAR": "0.00015605280000",
    "MEAN_MOTION_DOT": "0.000000000000",
    "SEMINOR_AXIS": "6796.863",
    "PERIOD": "92.944",
    "APOAPSIS": "421.852",
    "PERIAPSIS": "415.604",
    "OBJECT_TYPE": "PAYLOAD",
    "RCS_SIZE": "LARGE",
    "COUNTRY_CODE": "CIS",
    "LAUNCH_DATE": "1998-11-20",
    "SITE": "TMTTR",
    "DECAY_DATE": null,
    "FILE": "5255921",
    "GP_ID": "330713876",
    "TLE_LINE1": "0 ISS (ZARYA)",
    "TLE_LINE1": "1 25544U 98067A 26170.51159304 .00008252 00000-0 15605-3 0",
    "TLE_LINE2": "2 25544 51.6332 288.5889 0004596 205.0015 155.0751 15.4931585857"
  }
]
```

Note that it also mixes in data from the SATCAT with the GP data, includes elements for GP data that never change (e.g., "REF_FRAME": "TEME"), and then tacks on 3LE data, making it quite inefficient.

In fact, it takes the non-prettyfied [Space Track version](#) for [this example](#) from 422 characters to 1,134 characters for the equivalent [Space Track query—almost 3 times the size](#).

You can remove these unnecessary elements, but that means you need to individually specify which elements you *want* in each Space Track query (e.g., each of the 17 properties in the CelesTrak version). And you will still have numbers treated as strings.

Bottom line: If you use Space Track JSON data, you will need to adapt json2satrec() to use parseFloat(). It will ignore the unnecessary elements, but the csv2satrec(), which does the same thing, will need to either specifically select the same fields, in the same order as above, or modify the csv2satrec() code accordingly. The versions of csv2satrec() and json2satrec() discussed above expect to use the standard CelesTrak formats.

It turns out many of the calculations in SGP4 need only be performed once for any set of GP data and that is performed in the function sgp4init(). In fact, each of our format2satrec() functions end with that initialization:

```
// ----- initialize the orbit at sgp4epoch -----
sgp4init(satrec, {
  opsmode: opsmode,
  satn: satrec.satnum,
  epoch: satrec.jdsatepoch - 2433281.5,
  xbstar: satrec.bstar,
  xecco: satrec.ecco,
  xargpo: satrec.argpo,
  xinclo: satrec.inclo,
  xmo: satrec.mo,
  xno: satrec.no,
  xnodeo: satrec.nodeo
});
return satrec;
```

Our individual format converters simply plug the relevant parts of our data into a satrec for initialization. That allows us to make minimal modifications to the satellite-js code, which could lead to problems with our verification process. And you can quickly check that your CSV and JSON versions produced the same output.

Examples

Let's start with something simple. Let's start with a barebones JavaScript program that simply takes a GP data set and converts it to TEME output (position and velocity). We did this to verify our version of satellite-js against the official USSF version of SGP4. Which means you can use this example to validate your code (e.g., Python or MATLAB): [SGP4 Verification](#).

More to come soon...